

QNN

September 19, 2026

[]: Week-03(Day -11) QUANTUM NEURAL NETWORK-RAJIV KUMAR YADAV

The input features are encoded into qubits, while the trainable rotation angles act like weights in a neural network: The important difference from your previous QSVM program is that QSVM primarily uses a quantum kernel, whereas this QNN learns parameters of a parameterized quantum circuit.

```
[1]: import qiskit
import numpy as np

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

print("Qiskit:", qiskit.__version__)
print("NumPy:", np.__version__)

# =====
# 1. DATASET
# =====

X = np.array([
    [0.1, 0.2],
    [0.2, 0.1],
    [2.8, 2.9],
    [2.9, 2.8]
])

# Binary labels
y = np.array([0, 0, 1, 1])

# Convert features to quantum rotation angles
X_scaled = (X / 3.0) * np.pi

# =====
# 2. QUANTUM NEURAL NETWORK
# =====
```

```

def qnn_circuit(features, weights):

    qc = QuantumCircuit(2, 1)

    # -----
    # Input layer: encode classical features
    # -----

    qc.ry(features[0], 0)
    qc.ry(features[1], 1)

    # -----
    # Entanglement layer
    # -----

    qc.cx(0, 1)

    # -----
    # Trainable neural-network layer
    # -----

    qc.ry(weights[0], 0)
    qc.ry(weights[1], 1)

    qc.rz(weights[2], 0)
    qc.rz(weights[3], 1)

    # Additional entanglement
    qc.cx(0, 1)

    # -----
    # Measurement
    # -----

    qc.measure(0, 0)

    return qc

# =====
# 3. QUANTUM SIMULATOR
# =====

simulator = AerSimulator()

```

```

# =====
# 4. QNN PREDICTION
# =====

def qnn_predict(features, weights, shots=1000):

    qc = qnn_circuit(features, weights)

    compiled = transpile(qc, simulator)

    result = simulator.run(
        compiled,
        shots=shots
    ).result()

    counts = result.get_counts()

    probability_1 = counts.get("1", 0) / shots

    return probability_1

# =====
# 5. COST FUNCTION
# =====

def cost_function(weights):

    total_error = 0

    for features, label in zip(X_scaled, y):

        prediction = qnn_predict(
            features,
            weights
        )

        error = (prediction - label) ** 2

        total_error += error

    return total_error / len(y)

# =====
# 6. INITIAL PARAMETERS
# =====

```

```

np.random.seed(42)

weights = np.random.uniform(
    -np.pi,
    np.pi,
    4
)

print("\nInitial weights:")
print(weights)

print(
    "Initial cost:",
    cost_function(weights)
)

# =====
# 7. TRAINING
# =====

iterations = 100

best_cost = cost_function(weights)

print("\nTraining QNN...\n")

for iteration in range(iterations):

    # Create a small random change
    candidate_weights = (
        weights +
        np.random.normal(
            0,
            0.2,
            size=4
        )
    )

    candidate_cost = cost_function(
        candidate_weights
    )

    # Keep the new parameters
    # only if the cost improves
    if candidate_cost < best_cost:

```

```

        weights = candidate_weights
        best_cost = candidate_cost

    if iteration % 10 == 0:

        print(
            "Iteration:",
            iteration,
            "Cost:",
            round(best_cost, 4)
        )

# =====
# 8. FINAL PREDICTIONS
# =====

predictions = []
probabilities = []

for features in X_scaled:

    probability = qnn_predict(
        features,
        weights
    )

    probabilities.append(probability)

    # Convert probability into class
    if probability >= 0.5:
        predictions.append(1)
    else:
        predictions.append(0)

predictions = np.array(predictions)
probabilities = np.array(probabilities)

# =====
# 9. ACCURACY
# =====

accuracy = np.mean(

```

```

        predictions == y
    ) * 100

# =====
# 10. RESULTS
# =====

print("\n=====")
print("          QUANTUM NEURAL NETWORK")
print("=====")

print("\nFinal weights:")
print(weights)

print("\nActual labels:")
print(y)

print("\nPrediction probabilities:")
print(np.round(probabilities, 3))

print("\nPredicted labels:")
print(predictions)

print("\nAccuracy:")
print(round(accuracy, 2), "%")

print("\nFinal cost:")
print(round(best_cost, 4))

```

Qiskit: 2.5.2

NumPy: 2.4.6

Initial weights:

```
[-0.78828768  2.83192151  1.45766093  0.61988954]
```

Initial cost: 0.022742500000000006

Training QNN...

Iteration: 0 Cost: 0.0225

Iteration: 10 Cost: 0.0007

Iteration: 20 Cost: 0.0001

Iteration: 30 Cost: 0.0

Iteration: 40 Cost: 0.0

Iteration: 50 Cost: 0.0

Iteration: 60 Cost: 0.0

Iteration: 70 Cost: 0.0

Iteration: 80 Cost: 0.0

Iteration: 90 Cost: 0.0

```
=====
      QUANTUM NEURAL NETWORK
=====
```

Final weights:

[0.01636583 2.40596675 1.08466753 -0.26743625]

Actual labels:

[0 0 1 1]

Prediction probabilities:

[0.004 0.007 0.992 0.997]

Predicted labels:

[0 0 1 1]

Accuracy:

100.0 %

Final cost:

0.0

[]: